

FORMATION AI MASTERY

Construire un micro-SaaS automatisé

Architecture, exécution en 7 jours et automatisation par agents IA — du premier commit à la première facture Stripe, en dormant.

AVERTISSEMENT & CADRE DE LECTURE

Ce guide rassemble et synthétise des bonnes pratiques modern SaaS. Il ne constitue pas un conseil juridique, fiscal ou financier. Les références techniques citées renvoient aux documentations officielles de leurs éditeurs (Next.js, Supabase, Stripe, Vercel) — toujours valider la version la plus récente avant mise en production.

Toute reproduction partielle est autorisée à des fins pédagogiques, dans le cadre strict du programme AI Mastery. Les captures, extraits

de code et configurations sont fournis à titre indicatif : adaptez-les à votre contexte, votre juridiction et votre modèle d'affaires.

© AI MASTERY — TOUS DROITS RÉSERVÉS

Table des matières

Quatre chapitres pour transformer une intuition de produit en micro-SaaS rentable, déployé et entretenu par une équipe d'agents — pas d'employés.

CHAPITRE 01

Le concept moderne du micro-SaaS

Page 1

CHAPITRE 02

Architecture technique de référence

Page 4

CHAPITRE 03

Plan d'exécution en 7 jours

Page 9

CHAPITRE 04

Automatisation et support par agents IA

Page 12

ANNEXES

Variables, commandes, lectures

Page 16

Le concept moderne du micro-SaaS

Pendant vingt ans, lancer un logiciel payant signifiait lever des fonds, recruter une équipe et attendre dix-huit mois. En 2026, ce délai se mesure en jours — et l'équipe peut tenir dans un seul navigateur, à condition d'accepter une contrainte : choisir une douleur minuscule, et la résoudre radicalement.

Définition

Un **micro-SaaS** est un logiciel accessible par abonnement, conçu et maintenu par un seul individu (ou un duo fondateur), qui sert une audience étroite — souvent moins de 1 000 clients payants — sur une verticale si spécifique que les solutions généralistes l'ignorent. La recette unitaire faible mais récurrente, combinée à un taux de churn maîtrisé sur de petites cohortes, suffit à générer un revenu annuel récurrent (ARR) viable sans capitaux extérieurs.

Pourquoi ça marche · 5 raisons structurelles

1. **Charge utile extrême.** Une verticale étroite permet de retirer 90 % des fonctionnalités qu'un SaaS généraliste contient. Moins de surface = moins de bugs, support plus rapide, onboarding plus court.
2. **Time-to-market record.** Le triplet Next.js + Supabase + Stripe permet de mettre en ligne un produit facturable en moins de sept jours. La vitesse devient elle-même un avantage compétitif.
3. **Zéro capex.** Pas d'investisseurs, pas de board, pas de dilution. Le fondateur conserve 100 % de la valeur — et la liberté d'itérer sans permission.
4. **Souveraineté produit.** Cycle décisions → code → client extrêmement court. Quand un seul utilisateur se plaint, le patch peut sortir en moins d'une heure.

5. **Communautés verticales.** Forums Discord, newsletters, subreddits de niche concentrent les acheteurs. Une publication bien placée vaut un budget marketing de six chiffres.

Les frictions que le micro-SaaS résout

Le terrain de jeu du micro-SaaS, ce sont les frictions que les outils généralistes abandonnent en route. Cinq exemples réels, tous tenables par un seul développeur :

Newsletter analytics pour Substack

Les auteurs Substack ne peuvent pas tracker l'engagement par section d'e-mail. Un micro-SaaS peut injecter des pixels UTM invisibles et fournir un dashboard par sujet.

Calendrier éditorial pour podcasts vidéo

Les podcasteurs qui filment leurs sessions d'enregistrement veulent un rétroplanning tournage → montage → publication. Aucun géant n'attaque ce marché.

Signature de contrats freelance chrono

Plateforme ultra-simple pour freelancers francophones : upload modèle PDF, signature électronique probante, archivage légal — sans l'écosystème DocuSign.

OCR de tickets de caisse

Petites entreprises BTP/restauration qui veulent abattre leur TVA. Un OCR ciblé + export vers cabinet comptable en CSV mensuel.

Gestion d'abonnements Patreon

Créateurs qui veulent importer leurs tiers Patreon, mapper les perks et synchroniser avec Discord, sans la dette Slack + Zapier.

MARCHÉ · REPÈRES PRUDENTS

Revenu médian : autour de 1 200 € MRR au bout de 12 mois pour un micro-SaaS solo — variable selon la niche et l'effort marketing.

Taux de survie à 24 mois : proche de 30 % pour les projets bootstrappés sans validation préalable. Ceux qui interviewent au moins 8 prospects avant d'écrire la première ligne doublent ce taux.

ARR médian au terme de l'année 1 : entre 8 000 € et 45 000 € pour les 25 % supérieurs du panel. Au-dessus de 100 k € ARR, on bascule dans la catégorie « small SaaS ».

Anatomie d'un micro-SaaS réussi

Cinq caractéristiques opérationnelles :

- **Positionnement étroit.** On peut l'expliquer à un passant en moins de 30 secondes. Si l'explication requiert un diagramme, c'est trop large.
- **Payload minimal.** Trois à sept écrans, pas plus. Le reste est dette technique emballée dans du marketing.
- **Billing transparent.** Un seul prix visible, facturation Stripe Checkout. Pas de « contactez-nous pour un devis ». Le client doit pouvoir payer en moins de 60 secondes.
- **Support radical mais minimal.** Une boîte mail partagée, un délai affiché (« *réponse sous 12 heures ouvrées* »), zéro SLA mirobolant.
- **Stack standardisée.** Aucun composant custom inventé. Ce qui existe déjà sous forme de service managé doit y rester.

Anti-patterns · 4 pièges classiques

- **Over-ingénierie.** Kubernetes, microservices, file de messages distribuée pour 60 utilisateurs. La complexité tuera le produit avant qu'il n'atteigne 600 clients.
- **Niche trop large.** « Outils de productivité pour freelances » couvre 70 millions de personnes. Aucune douleur ne sera assez aiguë pour être facturée.

- **Prix dérivé à l'envers.** Fixer 49 € parce que « *le marché peut supporter* » au lieu de « *49 € est ce qui permet au client de gagner 200 € par mois sans hésiter* ». Le pricing doit être ancré dans la valeur perçue, pas le désir de revenu.
- **Fuite en avant sur le marketing.** Construire une landing page, lancer sur Product Hunt, acheter des Google Ads avant d'avoir un produit facturable cinq fois. Budget brûlé, retour nul.

POSITIONNEMENT

Si votre micro-SaaS peut être décrit en moins de **140 caractères** à un 陌生人, vous avez une verticale. Sinon, rebroussez.

Architecture technique de référence

Le triplet **Next.js + Supabase + Stripe** s'est imposé comme la référence 2026 du micro-SaaS pour une raison simple : chacun des trois services s'occupe d'une seule chose, ils sont tous les trois managés et facturés à l'usage, et leurs SDK convergent vers TypeScript. Cette section décrit l'architecture cible et les blocs de code prêts à coller.

Vue d'ensemble

Le navigateur parle à Next.js. Next.js parle à Supabase (données, auth) et à Stripe (paiements). Stripe parle à Next.js via webhook. Aucun service ne sort de cette boucle.

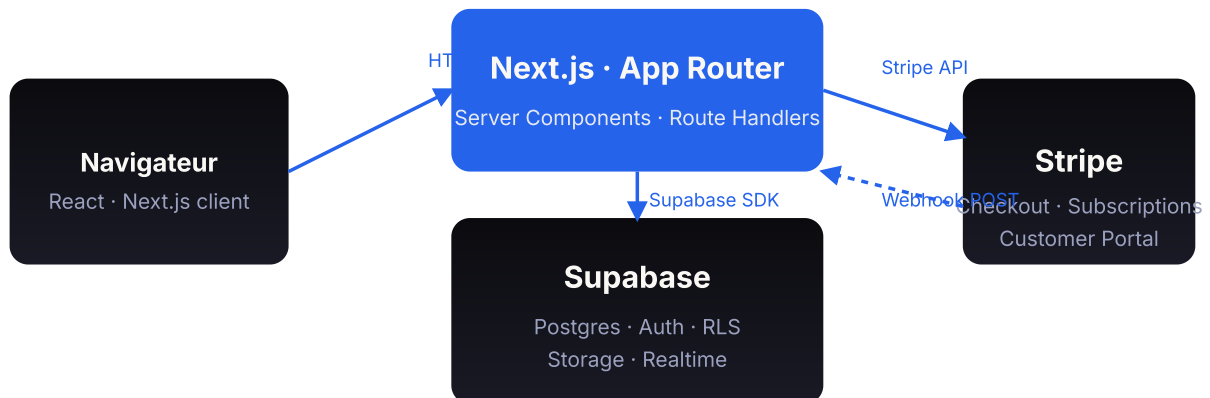


Figure 2.1 — Architecture cible d'un micro-SaaS Next.js + Supabase + Stripe.

Tableau comparatif des trois piliers

CRITÈRE	NEXT.JS	SUPABASE	STRIPE
Domaine	Front-end & back-end applicatif	Base de données & authentification	Paielements & facturation
Rôle dans le produit	UI React, SSR, API routes, logique métier	Postgres managé, Auth, Storage, Realtime	Checkout, abonnements, portail client
Service managé ?	Oui (Vercel)	Oui (Supabase Cloud)	Oui (Stripe SaaS)
Prix indicatif	0–20 \$/mois pour un micro-SaaS	0–25 \$/mois (Free tier très généreux)	2,9 % + 0,30 € par transaction
Points forts	App Router, Server Actions, Edge runtime	RLS native, dashboard SQL, Magic Link	Webhooks signés, Customer Portal, SCA-ready
Pièges à éviter	Server Actions pour webhook ≠ stable	Exposer la clé service_role côté client	Se fier à l'UI Stripe sans vérifier le webhook
Documentation	nextjs.org/docs	supabase.com/docs	docs.stripe.com

2.1 · NEXT.JS (APP ROUTER)

Next.js est la couche applicative universelle. L'App Router introduit quatre primitives à maîtriser :

- **Server Components** : rendu côté serveur, jamais exposés au navigateur. Idéal pour fetch direct Supabase.
- **Client Components** : traditionnels "use client", nécessaires dès qu'on a interactivité.
- **Server Actions** : fonctions RPC appelées depuis un form React. Idéales pour mutations internes.

- **Route Handlers** : véritables endpoints HTTP dans `app/api/.../route.ts`. Ce sont eux qui reçoivent les webhooks Stripe.

TECHNIQUE · WEBHOOK STRIPE

Règle non négociable : les webhooks Stripe doivent être implémentés dans un *Route Handler* (`route.ts`), jamais via une *Server Action*.

Pourquoi : une *Server Action* est une invocation RPC attachée à un form React — elle n'est pas un endpoint HTTP stable, son URL change à chaque build et ne respecte pas les conventions REST. Stripe attend un endpoint POST statique avec vérification de signature. La convention de fichiers `route.js` est documentée dans les (*Next.js docs – File convention route.js*).

BLOC CODE #1 — WEBHOOK STRIPE DANS ROUTE.TS

```
// app/api/stripe/webhook/route.ts
import { NextResponse } from "next/server";
import Stripe from "stripe";
import { createClient } from "@supabase/supabase-js";

export const runtime = "nodejs"; // Stripe SDK ≠ Edge

const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!, {
  apiVersion: "2024-06-20",
});

const supabase = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.SUPABASE_SERVICE_ROLE_KEY! // server-only
);

export async function POST(req: Request) {
  const sig = req.headers.get("stripe-signature");
  const body = await req.text(); // raw, jamais JSON.parse

  let event: Stripe.Event;
  try {
    event = stripe.webhooks.constructEvent(
      body, sig!, process.env.STRIPE_WEBHOOK_SECRET!
    );
  } catch (err) {
    return new NextResponse(`Webhook error: ${err}`, { status: 400 });
  }

  switch (event.type) {
    case "checkout.session.completed": {
      const session = event.data.object as Stripe.Checkout.Session;
      const userId = session.metadata?.user_id;
```

```

    if (userId) {
      await supabase.from("subscriptions").upsert({
        user_id: userId,
        stripe_customer_id: session.customer as string,
        stripe_subscription_id: session.subscription as string,
        status: "active",
        current_period_end: /* lu sur l'objet subscription */,
      });
    }
    break;
  }
  case "customer.subscription.updated":
  case "invoice.paid":
  case "invoice.payment_failed": {
    // ... sync status dans la table subscriptions
    break;
  }
}

return NextResponse.json({ received: true });
}

```

2.2 · SUPABASE (AUTH + POSTGRES + RLS)

Supabase fournit l'authentification (Magic Link, OAuth Google/GitHub), la base Postgres managée, un SDK unifié, et surtout — la **Row Level Security** (RLS), qui permet d'écrire la couche d'autorisation directement en SQL, près des données.

La RLS est **activée par défaut sur toute table du schéma public** dès sa création avec l'outil `supabase init` (*Supabase docs – Row Level Security*). C'est un garde-fou essentiel : sans RLS, n'importe quel utilisateur détenant la clé anon pourrait lire toutes les lignes d'une table.

BLOC CODE #2 — POLITIQUE RLS

```

-- migrations/0003_subscriptions_qls.sql

ALTER TABLE subscriptions ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users can read own subscriptions"
ON subscriptions
FOR SELECT
USING (auth.uid() = user_id);

CREATE POLICY "Service role can write all"
ON subscriptions
FOR ALL
USING (true)
WITH CHECK (true);
-- le service_role_key bypass RLS, mais on garde une policy explicite
pour la lisibilité et les audits.

```

SÉCURITÉ

Ne **jamais** exposer la clé `supabase_service_role` dans du code client. Elle contourne la RLS. Elle reste confined aux Route Handlers, Server Components (avec précautions) et migrations. Côté client, on n'utilise que la clé `NEXT_PUBLIC_SUPABASE_ANON_KEY`, qui respecte les politiques. Dans chaque policy, préférez `(select auth.uid())` à `auth.uid()` brut — la forme parenthésée permet à Postgres de mettre la valeur en cache, ce qui accélère les plans d'exécution.

2.3 · STRIPE (CHECKOUT, SUBSCRIPTIONS, WEBHOOKS)

Stripe remplit trois rôles distincts, à ne pas confondre :

- **Stripe Checkout** : page de paiement hébergée, SCA-ready, conforme aux normes internationales.
- **Customer Portal** : page self-service où l'utilisateur met à jour sa carte, change de plan, annule.
- **Webhooks signés** : source de vérité *asynchrone* pour votre base de données. Le front n'est jamais source de vérité sur l'état d'abonnement.

ÉVÉNEMENTS STRIPE À ÉCOUTER (EXTRAIT)

EVENT.TYPE	QUAND IL SURVIENT	ACTION DANS NOTRE CODE
checkout.session.completed	Paieement réussi d'un nouveau checkout	Provisionner l'accès (upsert active subscription)
customer.subscription.updated	Plan changé / renouvelé	Synchroniser current_period_end et status
invoice.paid	Paieement récurrent encaissé	Confirmer accès, journaliser la facture
customer.subscription.deleted	Abonnement annulé ou expiré	Révoquer l'accès au produit
invoice.payment_failed	Échec de paiement (carte refusée, etc.)	Notifier, ouvrir période de grâce

BLOC CODE #3 — SERVER ACTION « START CHECKOUT »

```
// app/(billing)/actions.ts
"use server";

import Stripe from "stripe";
import { createClient } from "@supabase/supabase-js";

const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!);

export async function startCheckout(priceId: string) {
  const supabase = createClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  );
  const { data: { user } } = await supabase.auth.getUser();
  if (!user) throw new Error("Not authenticated");

  const session = await stripe.checkout.sessions.create({
    mode: "subscription",
    customer_email: user.email,
    line_items: [{ price: priceId, quantity: 1 }],
    success_url: `${process.env.NEXT_PUBLIC_URL}/dashboard?ok=1`,
    cancel_url: `${process.env.NEXT_PUBLIC_URL}/pricing`,
    metadata: { user_id: user.id },
  });
}
```

```
    return session.url;  
}
```

Plan d'exécution en 7 jours

La règle est simple : **sept jours utiles, quatre à six heures chacun**, déployer en production le septième soir. Ce plan suppose que la verticale a été validée au préalable. Si vous n'avez pas encore interviewé de prospects, remplacez le J1 par ce travail — un produit construit sans peut s'effondrer sans bruit.

Philosophie ship-first

Au septième jour, le site doit accepter un vrai paiement en vrai euro. Pas un test. Pas un environnement de staging. Du vrai. Tout ce qui n'est pas strictement requis pour ce fait est poussé en V2.

Tableau du plan

JOUR	OBJECTIF	LIVRABLE CONCRET	OUTILS	INDICATEUR DE SUCCÈS
J1	Problème & marché	PRD d'une page, 8 interviews pros	Notion, Calendly, Loom	La douleur est nommée en moins de 140 caractères
J2	Prototype design	5 écrans cliquables, design system	v0.dev ou Figma	Maquette partageable à 3 prospects
J3	Bootstrap Next.js + Supabase	Repo initialisé, supabase init, RLS activée	Next.js, Supabase CLI, pnpm	Hello world déployé en preview Vercel
J4	Auth + tables	Magic Link Supabase, table users, table stripe_customers	Supabase Auth, SQL migration	Connexion bout-en-bout en local
J5	Stripe Checkout + webhook	Route handler webhook, state machine abonnement	Stripe CLI, stripe listen	Test payment en local, événement reçu en base
J6	Polish UX & landing	Page d'accueil, pricing, dashboard	Tailwind, Shadcn UI, copywriter AI	3 prospects cliquent sur Buy
J7	Déploiement Vercel + production	Build de production, env vars, DNS, 5 paiements test	Vercel CLI, Stripe Dashboard	URL publique, paiement réel validé

ASTUCE · LE PRD AVANT LE COMMIT

Numérotez votre PRD **avant** votre commit initial. Si vous ne pouvez pas nommer la douleur en moins de 140 caractères, rebroussez. Le code le plus rapide à écrire est celui qu'on évite d'écrire en entier.

TECHNIQUE · VARIABLES D'ENVIRONNEMENT VERCEL

Vercel distingue **Production** et **Preview** dans sa gestion d'environnement.

Toute variable ajoutée sans cible précise héritée est visible par les previews — y compris vos secrets Stripe et Supabase `service_role`.

En local, la commande documentée par Vercel (*Vercel docs – Environment variables*) est :

```
vercel env pull --environment production
```

— l'option `--environment` accepte aussi `preview` et `development`. Cela génère un `.env.local` aligné sur l'environnement cible. Ne commitez jamais ce fichier.

Pattern recommandé : secrets *uniquement* en Production
(`STRIPE_SECRET_KEY`, `SUPABASE_SERVICE_ROLE_KEY`, `STRIPE_WEBHOOK_SECRET`). Variables publiques (`NEXT_PUBLIC_*`) sur les trois environnements. Les Stripe webhook endpoints sont eux aussi séparés : un en local via `stripe listen`, un en preview via l'URL de preview Vercel, un en production.

Hygiène quotidienne

- Chaque commit doit compiler (`pnpm build`) en moins de 90 secondes.
- Chaque migration SQL est versionnée (`supabase/migrations/00XX_*.sql`).
- Aucune dépendance ajoutée sans note dans le PRD — la dette démarre vite.
- Une seule feature flag : `ENABLE_BILLING`. Si elle est `false`, Stripe est court-circuité — utile pour démos prospects sans les facturer.

Automatisation et support par agents IA

Un micro-SaaS bien construit ne se gère pas comme une startup — il se gère comme un service postal. Le fondateur dort. Quand il se réveille, les e-mails ont été triés, les nouveaux comptes activés, les billets à risque flagués, et un article de fond a été publié sur le blog. Cette section décrit l'équipe d'agents minimale qui rend cela possible.

Vision

Quatre agents, quatre responsabilités, un plafond ferme sur ce qu'ils peuvent faire sans validation humaine. L'idée principale : l'IA traite 95 % du volume, un humain supervise 5 % à valeur ajoutée. Cette proportion n'est pas une métrique d'org chart — c'est la différence entre burnout et sommeil.

Les 4 agents autonomes

AGENT 01

Support (inbox drafts)

Déclencheur : nouvel e-mail entrant sur support@tonsaas.com.

Comportement : lit le message, interroge Postgres sur le compte client, rédige une réponse contextualisée, classe dans une file « drafts ».

Outils : GPT-4o (ou modèle équivalent) avec tool use, retrieval sur la base, accès read-only au compte Stripe du client.

Garde-fou humain : oui, obligatoire — humain valide ou réécrit avant envoi. Pattern *human-in-the-loop*.

AGENT 02

Onboarding (sequences & alertes)

Déclencheur : nouvel utilisateur + J+1, J+3, J+7 calendaires.

Comportement : envoie une séquence d'e-mails adaptée, observe l'activation (a-t-il créé son premier projet ?). Si stagnation à J+3, alerte Slack du fondateur.

Outils : cron Vercel, Resend ou Postmark.

Garde-fou humain : **oui, sur les alertes** — fondations en automatique, escalade sur stagnation.

AGENT 03

Churn-Watch

Déclencheur : quotidien à 06:00 UTC.

Comportement : croise Stripe (statut paiement) + event log (usage 7 derniers jours) + tickets support ouverts. Calcule un score de risque ; pousse les comptes à risque moyen dans une table alerts.

Outils : cron Vercel, requêtes Supabase, table alerts.

Garde-fou humain : **non pour la détection**, oui pour l'action : le fondateur choisit s'il envoie un e-mail manuel, une remise, ou rien.

AGENT 04

Content-Marketing

Déclencheur : hebdomadaire dimanche 23:00 UTC.

Comportement : scrape tendances de la verticale (Reddit, HN, X), génère un article long-format signé du fondateur avec citations, publie dans le blog, indexe pour SEO, partage sur la newsletter.

Outils : Perplexity + Claude pour la rédaction, Sanity ou MDX pour la publication.

Garde-fou humain : **faible, mais revue éditoriale**. Le fondateur lit le dimanche matin, annule ou édite avant publication effective.

Pattern de sécurité : humain dans la boucle

Règle d'or : un agent ne doit jamais modifier un état financier (annulation, refund, prorata) sans validation explicite. Le webhook Stripe est notre source de vérité ; tout agent qui propose une action sur Stripe doit passer par une *draft action* que le fondateur valide via une UI d'administration.

BLOC CODE #4 — CRON VERCEL POUR L'AGENT CHURN-WATCH

```
// vercel.json
{
  "crons": [
    {
      "path": "/api/agents/churn-watch",
      "schedule": "0 6 * * *"
    },
    {
      "path": "/api/agents/onboarding-tick",
      "schedule": "0 9 * * *"
    },
    {
      "path": "/api/agents/content-weekly",
      "schedule": "0 23 * * 0"
    }
  ]
}
```

BLOC CODE #5 — HANDLER DE L'AGENT CHURN-WATCH

```
// app/api/agents/churn-watch/route.ts
import { NextResponse } from "next/server";
import { createClient } from "@supabase/supabase-js";

export const runtime = "nodejs";

export async function GET() {
  const supabase = createClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.SUPABASE_SERVICE_ROLE_KEY!
  );

  const { data: atRisk } = await supabase
    .rpc("churn_scan", { days_lookback: 7 });

  for (const user of atRisk ?? []) {
    if (user.usage_ratio < 0.30) {
```

```

    await supabase.from("alerts").insert({
      user_id: user.id,
      kind: "low_usage",
      score: 1 - user.usage_ratio,
      metadata: { source: "churn-watch", ran_at: new Date().toISOString() },
    });
  }
}

return NextResponse.json({ scanned: atRisk?.length ?? 0 });
}

```

GARDE-FOUS · MÉTRIQUES & PLAFOND

Mesurez chaque mois :

- **CSAT proxy** : % de réponses support envoyées sans retouche humaine (cible > 70 %).
- **Taux de désabonnement post-intervention** : chiffre négatif = l'agent marche, positif = il dégrade l'expérience.
- **Hallucinations support** : une fois par trimestre, échantillonnez 20 réponses et notez la fidélité aux faits.

Plafond d'automatisation ferme : **aucune action irréversible** (annulation, refund, changement de plan) sans validation humaine.

Stack d'automatisation recommandée

OUTIL	USAGE	COÛT MENSUEL ESTIMÉ	OPEN-SOURCE ?	ALTERNATIVE
Vercel Cron	Schedules simples (notre cas)	0–20 \$	Non	Inngest, Trigger.dev
Inngest	Workflows durables, retries, steps	0–25 \$	Non	Trigger.dev, Temporal
Resend / Postmark	Envoi e-mails transactionnels & séquences	0–20 \$	Non	Sendgrid, Mailgun
LLM (GPT-4o / Claude)	Support, content	20–80 \$	Non	Ollama + Mistral (self-host)
Ollama + Mistral	Self-host LLM, RGPD-friendly	Serveur dédié	Oui	vLLM, llama.cpp
Browserless / Playwright	Scraping Content-Marketing	0–30 \$	Oui (Playwright)	Apify

Risques & éthique

- **Transparence.** Mention explicite dans les e-mails : « *Réponse assistée par IA — un humain relira sous 24 heures* ». Aucune simulation d'humanité.
- **RGPD.** Logs minimisés. Opt-in explicite pour toute analyse conversationnelle. Conservation 90 jours max, purge automatique côté Supabase.
- **Hallucination.** Le LLM agent n'a accès qu'en *read-only* à la base et à Stripe. Pas d'écriture financière. Sortie structurée (JSON schema) pour limiter les dérives.
- **Dépendance fournisseur.** Prévoir un fallback LLM + un fallback e-mail. Si OpenAI tombe, le support continue en mode dégradé (file manuelle).

Annexes

A · Variables d'environnement complètes

VARIABLE	VISIBILITÉ	ENVIRONNEMENT	SOURCE
NEXT_PUBLIC_SUPABASE_URL	Publique	Prod · Preview · Dev	Supabase Dashboard > Settings > API
NEXT_PUBLIC_SUPABASE_ANON_KEY	Publique	Prod · Preview · Dev	Supabase Dashboard > Settings > API
SUPABASE_SERVICE_ROLE_KEY	Secrète (server-only)	Prod uniquement	Supabase Dashboard > Settings > API
STRIPE_SECRET_KEY	Secrète (server-only)	Prod · Preview · Dev (avec clés test)	Stripe Dashboard > Developers > API keys
STRIPE_WEBHOOK_SECRET	Secrète (server-only)	Prod · Preview · Dev (un secret par endpoint)	Stripe Dashboard > Webhooks
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY	Publique	Prod · Preview · Dev	Stripe Dashboard > Developers > API keys
NEXT_PUBLIC_URL	Publique	Prod · Preview · Dev	URL absolue du déploiement

B • Commandes utiles (à copier-coller)

- `pnpm create next-app@latest mon-saas --ts --tailwind --app --src-dir`
- `npx supabase init` puis `npx supabase link --project-ref <ref>`
- `supabase db push` — appliquer les migrations locales au projet distant
- `stripe listen --forward-to localhost:3000/api/stripe/webhook` — tunnel CLI Stripe
- `stripe trigger checkout.session.completed` — tester un événement
- `vercel env pull --environment production` — récupérer les secrets prod localement
- `vercel --prod` — promouvoir la branche courante en production

C • Lectures recommandées

- [Next.js docs — File convention route.js](#)
- [Supabase docs — Row Level Security](#)
- [Stripe docs — Subscription webhooks](#)
- [Vercel docs — Environment variables](#)
- [Supabase RLS — patterns de production](#) (MakerKit)

AI Mastery · Officiel

De l'idée à la première facture, en dormant.

Ce guide a couvert quatre fronts parallèles : la discipline du positionnement, la stérilité d'une architecture standardisée, la rigueur d'un plan de sept jours, et l'élégance d'une équipe composée d'agents IA. Maîtrisés conjointement, ils produisent un type rare d'entreprise : une boîte qui dort.

RÉCAPITULATIF DES QUATRE CHAPITRES

Chapitre 1 · Le concept moderne — un seul fondateur, une verticale étroite, une douleur nommée en 140 caractères. Tout le reste est secondaire.

Chapitre 2 · Architecture de référence — Next.js + Supabase + Stripe. Webhooks via Route Handlers. RLS partout. Secrets en server-only.

Chapitre 3 · Plan d'exécution en 7 jours — sept jours utiles, livraison publique le septième soir, paiement réel validé.

Chapitre 4 · Agents IA — Support, Onboarding, Churn-Watch, Content-Marketing. Humain dans la boucle sur tout ce qui est financier.

AI MASTERY · FORMATION OFFICIELLE

GUIDE DE RÉFÉRENCE · CONSTRUIRE UN MICRO-SAAS AUTOMATISÉ
ÉDITION 2026 · VOLUME 1 · V1.0